



GITHUB COPILOT

Model Selection

Right Model, Right Task

20 Minutes

Developers

Architects

Engineering Leaders



The 3-Step Framework

Classify the task, identify constraints, select the tier



The Model Landscape

GPT-5.5, Claude Opus 4.7, Gemini 3.1 — strengths and tradeoffs



Model-Task Matrix

Which model wins for which task



Tier Strategy & Cost

Real-world cost examples and savings



Practical Patterns

Fallback chains, task decomposition, evaluation



What You Can Do Today

Actionable next steps for this afternoon, week, and month



"With dozens of frontier models available, how do I choose the right one for each task — and when does it actually matter?"



Cost

40x price difference across tiers



Quality

10-25% accuracy gap on hard tasks



Speed

10x latency variation

One-Model-Fits-All is Expensive

Teams default to the "best" model for everything — paying premium prices for tasks where a fast model produces identical results. \$2,400/mo → \$480/mo with routing

Wrong Model = Wrong Results

A speed-optimized model on complex reasoning produces shallow, error-prone output. Haiku on a 15-microservice refactor → 12 bugs. Opus → zero regressions

No Framework for Decisions

Without clear criteria, teams guess or follow hype — neither scales across an engineering organization with dozens of use cases

The Landscape Changes Quarterly

New models ship constantly. Yesterday's best pick may not be today's — you need a repeatable process, not a static answer

You need a framework that works regardless of which models are on top this quarter

A 3-Step Framework

-  **Step 1: Classify the task** — What are you actually asking the model to do?
-  **Step 2: Identify the constraint** — What matters most for THIS task?
-  **Step 3: Select the tier** — Match to the capability tier that fits



Capability

Complex reasoning, architecture, security.
Highest quality, highest cost.



Balanced

Daily coding, writing, general tasks. Best cost-quality tradeoff.



Speed / Cost

Classification, triage, log parsing. High volume, low complexity.

Provider-agnostic — this framework works regardless of which models ship next quarter.

What are you actually asking the model to do? Seven categories:

 Reasoning

Multi-step logic, abstract problem-solving

Architecture decisions, root cause analysis

 Coding

Code generation, refactoring, debugging

Multi-file changes, test generation

 Writing

Prose, brand voice, documentation

Blog posts, user-facing copy

 Classification

Categorization, triage, labeling

Ticket routing, sentiment analysis

 Multimodal

Vision, audio, video processing

Image analysis, diagram interpretation

 Analysis

Long-document processing, data extraction

Contract review, log analysis

 Agentic

Multi-step, tool-using, autonomous workflows — distinct because reliability and instruction-following matter more than raw benchmark scores

Issue-to-PR, CI pipeline debugging, multi-step research

What matters most for THIS specific task? Resolve deployment first — it narrows your candidate list:

 Deployment Model **RESOLVE FIRST**

Public API / private cloud (Bedrock, Vertex, Azure) / self-hosted? This determines what's even available based on data residency and compliance.

 Accuracy

What's the cost of a wrong answer? High-stakes = premium tier, low-stakes = fast tier.

 Latency

Does it need to respond in real-time? Interactive UX demands fast models.

 Cost / Volume

10 requests/day or 10,000? Volume changes the math completely.

 Context Size

How much input at once? 4K, 128K, 1M, 2M — each unlocks different strategies.

 Safety

Security-sensitive or handling PII? Prompt injection resistance becomes critical.

Match your task + constraint to a vendor-neutral capability tier (not a pricing row):

 Capability Tier

Complex reasoning, architecture, security review, multi-file refactoring

Opus 4.7 · GPT-5.5
as of May 2026

 Balanced Tier

Daily coding, writing, general tasks — strong across most tasks

Sonnet 4.6 · GPT-5.4
as of May 2026

 Speed / Cost Tier

Classification, triage, log parsing, simple code — high volume

Haiku 4.5 · GPT-5.4 mini
as of May 2026

 Self-Hosted Tier

Data sovereignty, no per-token cost, full control — evaluate, don't assume quality

Gemma 4 · Llama 4
as of May 2026

⚠️ Label specific model names as "example as of [date]" — the landscape shifts quarterly.

Stop asking "what's the best model?"

Start asking "what's the right model for this task?"

✗ Before

One premium model for everything
\$\$\$, slow, diminishing returns

✓ After

Route tasks to the right tier
Same quality, 60-80% lower cost



The Model Landscape

Three providers, three philosophies, March 2026

OpenAI — GPT-5

GPT-5.5 NEW

Best agentic workflows, 1M context, omnimodal, 74 tok/sec

GPT-5.4

Top math reasoning (100% AIME), 83% GDPval knowledge

GPT-5.4 mini

Fast/cheap tier, 0.33x request cost

🏆 Best at: Agentic, reasoning, speed

Anthropic — Claude 4

Opus 4.7 NEW

1M context, best agentic chaining, 15x premium request cost

Sonnet 4.6

Best writing, most natural prose, 1x cost

Haiku 4.5

Fast triage, classification, 0.33x cost

🏆 Best at: Coding, safety, precision

Google — Gemini 3.1

Gemini 3.1 Pro NEW

2M context, 2x reasoning vs 3.0, best multimodal

Gemini 3.1 Flash

Ultra-fast, lowest \$/token, Copilot-supported

Gemma 4 (open)

Leading open-weight model for on-premise

🏆 Best at: Multimodal, context, cost

Dimension	GPT-5.5	Claude Opus 4.7	Gemini 3.1 Pro
Agentic / Multi-step	🏆 Best autonomy	🏆 Best chaining	🏆 Good
Coding (SWE-bench)	🏆 ~78%	🏆 83%+	🏆 ~72%
Math / Reasoning	🏆 100% AIME	🏆 Strong	🏆 2x improved
Context Window	1M	1M	🏆 2M
Multimodal	🏆 Omnimodal	🏆 Text-focused	🏆 Best vision+audio
Speed (tokens/sec)	🏆 74 tok/s	🏆 50 tok/s	🏆 Fast
Safety / Injection	🏆 Strong	🏆 Lowest risk	🏆 Good
Cost (\$/M tokens)	🏆 Moderate	🏆 Premium (15x)	🏆 Lowest

💡 **No single model wins every row.** That's exactly why task-based selection matters — the "best" model depends entirely on what you're optimizing for.



Model-Task Matrix

Which model wins for which task — backed by benchmarks

Task	Top Pick	Runner-Up	Why This One Wins
Code generation & refactoring	Claude Opus 4.7	GPT-5.5	83% SWE-bench, best multi-file accuracy
Agentic multi-step workflows	GPT-5.5	Claude Opus 4.7	Best autonomy, tool-use, omnimodal
Mathematical reasoning	GPT-5.5	Gemini 3.1 Pro	100% AIME, top ARC-AGI-2
Long-document analysis	Gemini 3.1 Pro	Claude Opus 4.7	2M token native context window
Multimodal (vision/audio)	Gemini 3.1 Pro	GPT-5.5	Best image/video/audio integration
Writing & brand voice	Claude Sonnet 4.6	GPT-5.5	Human-preferred, most natural prose
Security-sensitive tasks	Claude Opus 4.7	GPT-5.5	Lowest prompt injection rate
Speed & cost at scale	Gemini 3.1 Flash	GPT-5.4 mini	Fastest throughput, lowest \$/token
Data extraction	GPT-5.5	Gemini 3.1 Pro	Strongest structured output

Key takeaway: Each provider leads in 3 categories. There is no universal winner — only task-specific champions.



Decision Framework

Tier strategy and real-world cost examples



Premium Tier

Opus 4.7 · GPT-5.5

Complex reasoning, architecture, security review, multi-file refactoring. High \$/token but prevents costly mistakes.



Standard Tier

Sonnet 4.6 · GPT-5.4 · Gemini 3.1 Pro

Daily coding, writing, general tasks, document analysis. Balanced cost and quality for 60% of workloads.



Fast / Cheap Tier

Haiku 4.5 · GPT-5.4 mini · Gemini 3.1 Flash

Classification, triage, log parsing, simple code, bulk processing. 80% of volume should run here.

10,000 Support Ticket Classification

✗ Opus 4.6

\$500 · 8 hours · 99.2%

✓ Haiku 4.5

\$12 · 45 min · 98.8%

💡 0.4% accuracy gap doesn't justify 40x cost

Auth Refactor Across 15 Microservices

✗ Haiku 4.5

Fast · Misses deps · 12 bugs

✓ Opus 4.6

Slower · Maps all deps · 0 bugs

💡 Premium model pays for itself in avoided debugging

Match the model cost to the **cost of getting it wrong**



Practical Patterns

Three proven strategies for model routing

1 Fallback Chains

Try fast first, escalate on failure

Haiku → Sonnet → Opus

80% of requests handled at fast-tier pricing.
Premium only for the hard 20%.

Best for: High-volume APIs

2 Task Decomposition

Split complex work by tier

- 🏆 Planning → Premium
- ⚡ Implementation → Standard
- 🚀 Tests/docs → Fast

Each subtask gets the tier it deserves — no overspend, no quality gap.

Best for: Complex projects

3 Evaluation-Driven

Benchmark on YOUR data

1. 50-100 examples from your domain
2. Run 2-3 candidate models
3. Score on YOUR criteria
4. Choose the cost/quality sweet spot

Best for: Production deployment

How It Works

1. Route every request to the **fast tier** first
2. Evaluate confidence score of response
3. If below threshold → retry with **standard tier**
4. If still uncertain → escalate to **premium tier**

When to Use

High-volume APIs, customer support, code review triage, log analysis — anywhere 80%+ of requests are routine

Example Flow

→ Haiku 4.5

✓ confidence 0.92 → return

→ Haiku 4.5

✗ confidence 0.61 → escalate

→ Sonnet 4.6

✓ confidence 0.88 → return

🔔 Result

80% at fast-tier pricing, 15% standard, only 5% premium. ~70% cost reduction vs always-premium.

How It Works

1. **Analyze** incoming task complexity
2. **Decompose** into subtasks by tier need
3. **Route** each subtask to appropriate model
4. **Compose** results into final output

When to Use

Complex projects with mixed requirements — agent orchestration, PR reviews, feature development where planning ≠ implementation ≠ testing

Feature Build Example

🏆 **Opus 4.7** → Architecture plan, API design

⚡ **GPT-5.4** → Implementation, code generation

🚀 **Haiku 4.5** → Tests, docs, boilerplate

📌 Result

Premium only for the 10% that needs it. **~60% savings** vs premium-for-all, no quality gap where it counts.

How It Works

1. Curate **50-100 examples** from your domain
2. Define scoring rubric (accuracy, latency, cost)
3. Run **2-3 candidate models** on same inputs
4. Pick the **cost/quality sweet spot**
5. Re-run quarterly as models evolve

When to Use

Production deployment decisions, high-stakes use cases, compliance environments — anywhere you need data-backed model selection, not vibes

Sample Eval Results

Opus 4.7	97.2% · \$4.80 · 3.1s
GPT-5.5	95.8% · \$1.20 · 1.4s
Haiku 4.5	91.4% · \$0.08 · 0.3s

💡 Insight

Public benchmarks ≠ your workload. A 1.4% accuracy gap might justify **4x cost savings** — or might be unacceptable. Only YOUR data tells you.

✓ Move Toward

Task-based model routing

Cost-quality tradeoff analysis

Benchmark on YOUR data, not leaderboards

Fallback chains: fast first, escalate

Quarterly re-evaluation cycle

● Move Away From

One premium model for everything

Choosing based on hype or brand

Ignoring cost until the bill arrives

Trusting public benchmarks blindly

"Set and forget" model selection

Example: Before: \$2,400/month using Opus for everything. After: Routing 80% to Haiku/Sonnet, Opus only for architecture and security
→ \$480/month, same quality where it matters.

This Afternoon

- Audit current model usage: what tasks → which models?
- Identify top 3 cost-heavy tasks
- Check: could a cheaper model handle them?

This Week

- Build 50-example eval set from your workload
- Benchmark 2-3 models on cost, quality, latency
- Implement one fallback chain for highest-volume task

This Month

- Deploy model routing across team
- Set up cost monitoring per task category
- Schedule quarterly model review

Benchmarks & Comparisons

-  [LM Council Benchmarks \(March 2026\)](#)
-  [GPT vs Claude vs Gemini Comparison](#)
-  [GPT-5.4 vs Opus 4.6 vs Gemini 3.1 Pro](#)

Related Talks

- [Agent Teams](#) — Per-agent model selection in Squad
- [Enterprise Patterns](#) — Org-wide model governance
- [Context Engineering](#) — Maximizing model effectiveness



Model Selection

Right Model, Right Task

CopilotTraining · Tech Talk